

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN

Bài giảng môn học: HỆ ĐIỀU HÀNH

Bài 8:

**Quản lý thiết bị I/O
(Input/Output)**

Số tín chỉ: 3
Số tiết: 60 tiết
(30 LT + 30 TH)

Biên soạn: ThS. Nguyễn Văn Thành
Email : thanhnv@ntt.edu.vn

Bài 8: Quản lý thiết bị I/O

Thiết bị nhập / xuất (Input/Output)

Giao tiếp vật lý với thiết bị I/O

Giao tiếp bằng chương trình

I/O interface cho ứng dụng

I/O Subsystem của Hệ điều hành

Hiệu suất của I/O

Thiết bị nhập / xuất (Input/Output)

■ Thiết bị I/O:

- ▶ Thiết bị nhập / xuất (*Input / Output devices*): những thiết bị có khả năng nhập / xuất dữ liệu với hệ thống máy tính.
 - **Thiết bị Xuất** (*Output devices*): thiết bị nhận dữ liệu *xuất ra* từ hệ thống máy tính (*printer, monitor...*)
 - **Thiết bị Nhập** (*Input devices*): thiết bị đưa dữ liệu *nhập vào* hệ thống máy tính (*keyboard, mouse, scanner...*)
- ▶ Mỗi thiết bị I/O có thể có:
 - Chip xử lý (*processor*), thanh ghi (*registers*) riêng.
 - Bộ nhớ (*memory*) riêng.
 - Đường Bus riêng.
 - Chương trình điều khiển (*controller*) riêng.

Thiết bị nhập / xuất (Input/Output)

■ Phân nhóm thiết bị theo hình thức giao tiếp:

▶ *Character devices* (thiết bị ký tự)

- Lệnh và dữ liệu truyền đi theo từng dòng ký tự (*character*).
- Tác vụ: *get*, *put*..
- Thiết bị: *keyboard*, *mouse*, *printer*...

▶ *Block devices* (thiết bị khối)

- Lệnh và dữ liệu truyền đi theo từng khối (*block*).
- Tác vụ: *read*, *write*, *seek*...
- Thiết bị: *hard disk*, *floppy disk*, *removable disk* CD/DVD

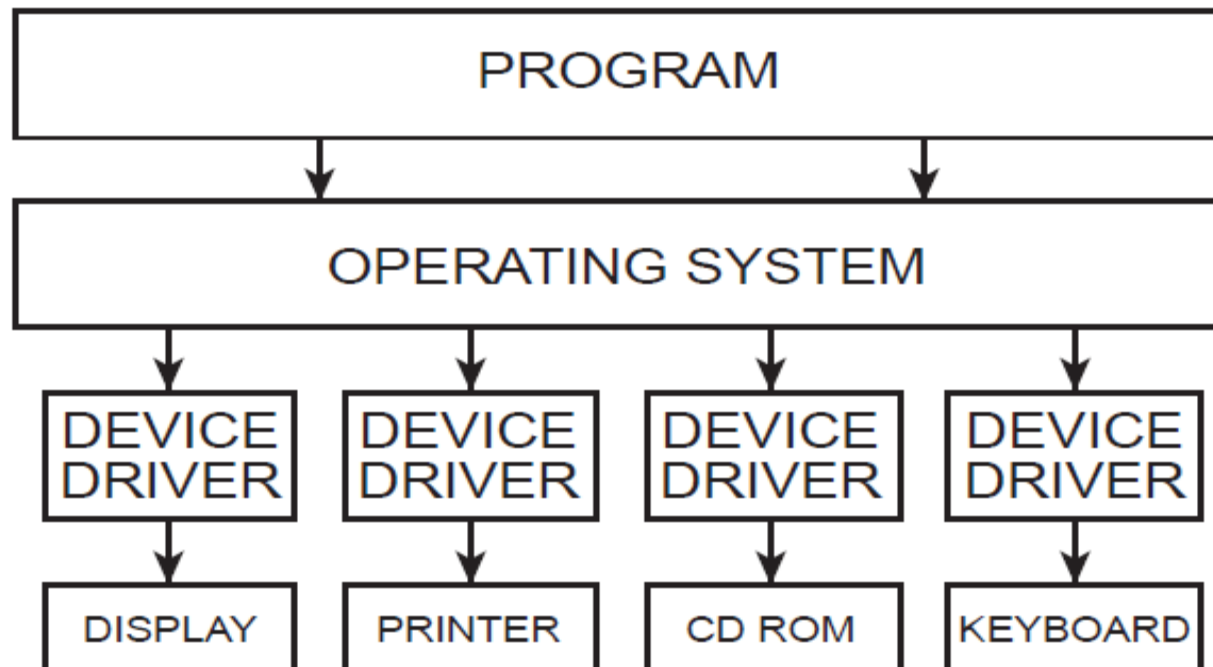
▶ *Network devices* (hay *Socket devices*)

- Chương trình tạo ra một *socket* và gán địa chỉ *Port* (TCP/UDP).
- Lệnh và dữ liệu được đưa vào *socket* để truyền sang *socket* máy khác.
- Thiết bị: *card giao tiếp mạng*

Thiết bị nhập / xuất (Input/Output)

■ Trình điều khiển thiết bị (*Device Driver*):

- ▶ *Driver* là một chương trình điều khiển hoạt động của thiết bị.
- ▶ Hệ điều hành *giao tiếp, truy xuất, điều khiển...* thiết bị thông qua *Driver* của thiết bị đó.



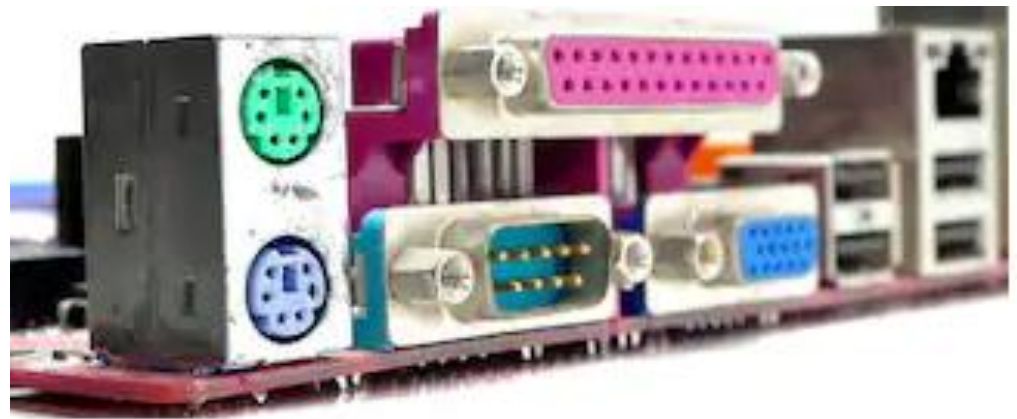
Giao tiếp vật lý với thiết bị I/O

Kết nối vật lý giữa thiết bị ngoại vi và hệ thống máy tính bằng 1 trong 3 hình thức:

- Dùng **Port** (cổng kết nối).
- Dùng **Bus** (đường truyền)
- Dùng **Controller** (bộ điều khiển)

■ Giao tiếp dùng **Port** vật lý:

- *PS/2 port*: cổng giao tiếp Keyboard, mouse
- *Serial port*: cổng COM
- *Parallel port*: cổng LPT.



Giao tiếp vật lý với thiết bị I/O

■ Giao tiếp dùng *Controller*:

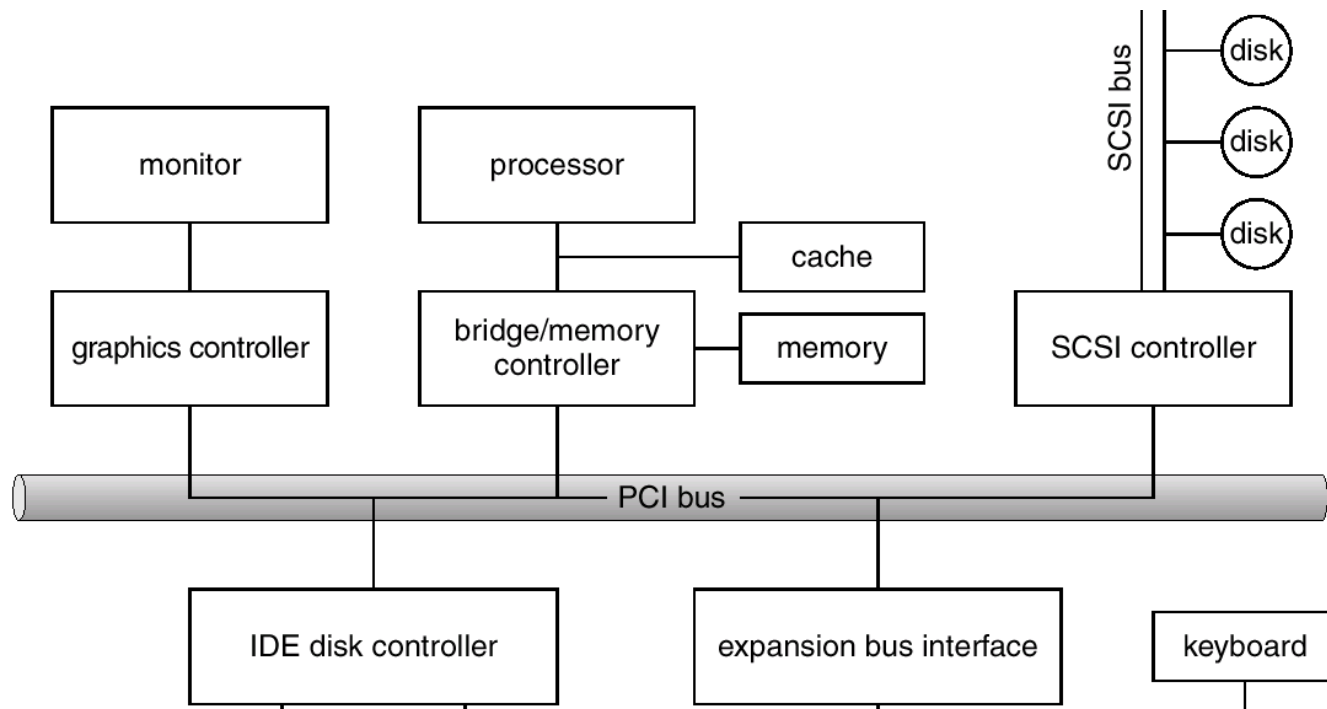
- ▶ *IDE controller*: bộ điều khiển giao tiếp HDD / CD / DVD
- ▶ *SCSI controller*: bộ điều khiển giao tiếp HDD chuyên dụng
- ▶ *Ethernet controller*: bộ điều khiển giao tiếp mạng.
- ▶ *Graphic controller*: bộ điều khiển VGA (card màn hình)
- ▶ *Sound controller*: bộ điều khiển âm thanh.



Giao tiếp vật lý với thiết bị I/O

■ Giao tiếp dùng *Bus*:

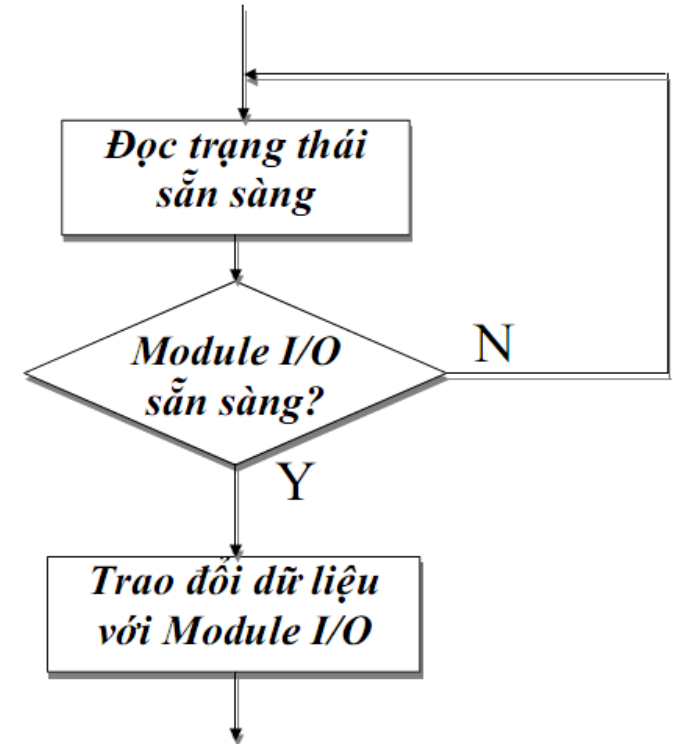
- ▶ *PCI bus*: giao tiếp các thiết bị bên trong hệ thống máy tính.
- ▶ *SCSI bus*: giao tiếp với các disks trong *SCSI controller*.
- ▶ *USB*: bus giao tiếp thiết bị ngoại vi đa năng.



Giao tiếp bằng chương trình

■ Dùng kỹ thuật điều khiển *Polling*:

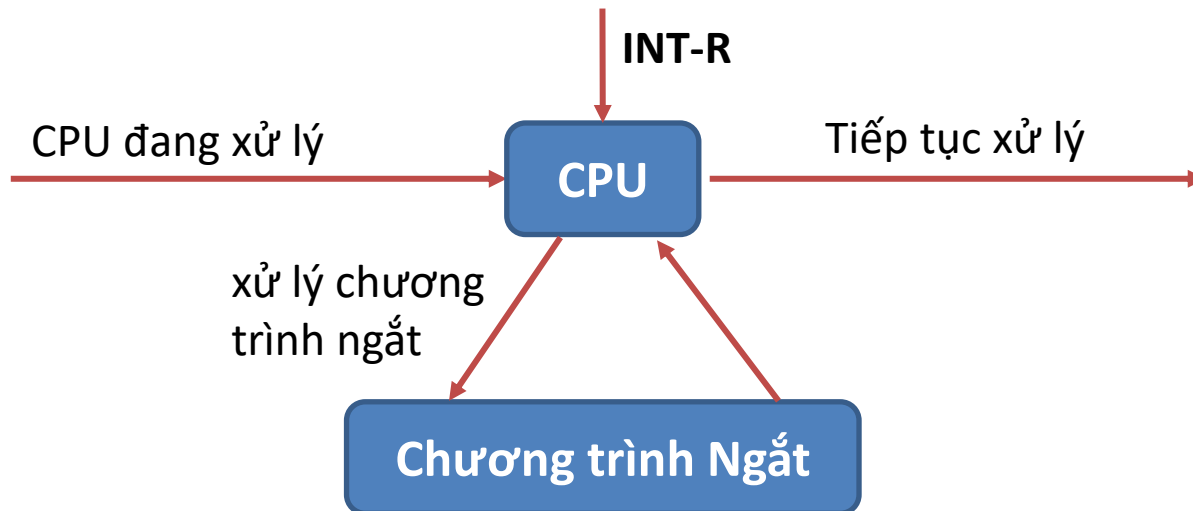
- ▶ *Polling* (thăm dò): là kỹ thuật thăm dò trạng thái của thiết bị I/O trước khi giao tiếp.
- ▶ Cơ chế: (xem sơ đồ mô tả)
- ▶ Kỹ thuật *Polling* thực hiện khi *process* có yêu cầu truy cập I/O => *Polling* là kỹ thuật dạng *Programmed I/O*.



Giao tiếp bằng chương trình

■ Dùng kỹ thuật *interrupt* (ngắt)

- ▶ Kỹ thuật *Interrupt* thực hiện khi *thiết bị I/O* phát yêu cầu giao tiếp với hệ thống..
- ▶ Cơ chế:
 - Khi có nhu cầu, thiết bị I/O gửi tín hiệu **IRQ** (*Interrupt Request*) yêu cầu CPU tạm dừng công việc để trao đổi dữ liệu với thiết bị.
 - CPU sẽ tạm dừng tiến trình để phục vụ chương trình ngắt.



Giao tiếp bằng chương trình

■ Dùng kỹ thuật *interrupt* (ngắt)

▶ *Bảng Vector ngắt*:

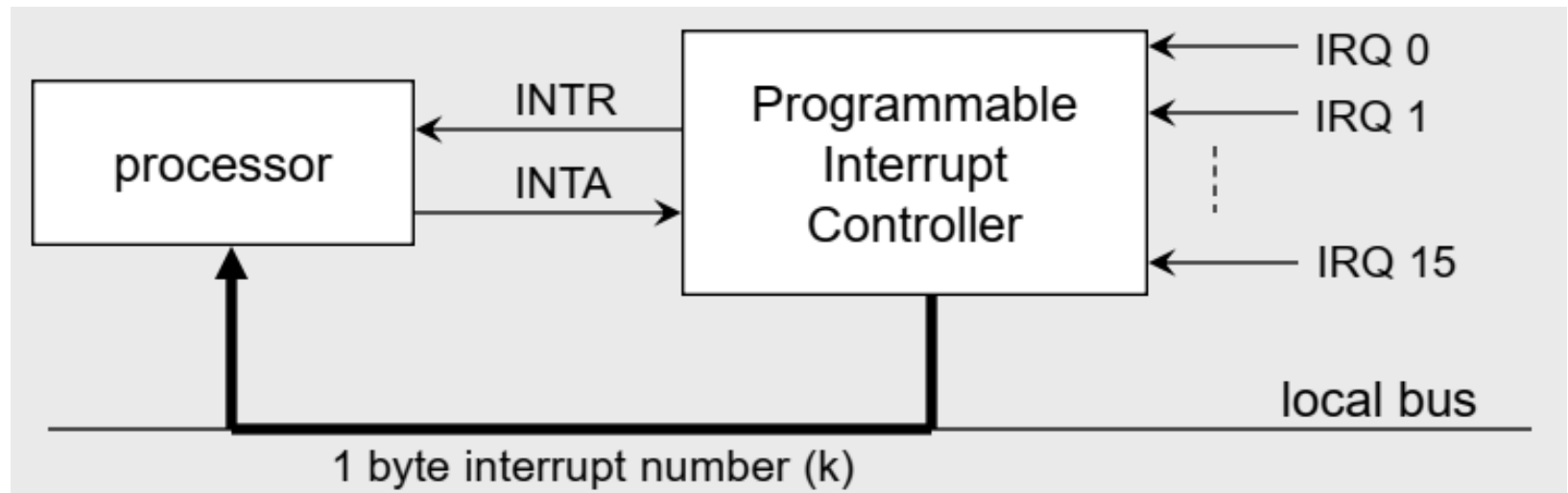
- Có nhiều thiết bị I/O, mỗi thiết bị có chương trình ngắt riêng.
- => Mỗi chương trình ngắt cần có mã IRQ riêng (*IRQ number*).
- *Bảng vector ngắt* chỉ định *IRQ number* và chương trình ngắt tương ứng với thiết bị..

IRQ	Usage
0	system timer (cannot be changed)
1	keyboard controller (cannot be changed)
2	cascaded signals from IRQs 8–15
3	second RS-232 serial port (COM2: in Windows)
4	first RS-232 serial port (COM1: in Windows)
5	parallel port 2 and 3 or sound card
6	floppy disk controller

Giao tiếp bằng chương trình

■ Dùng kỹ thuật *interrupt* (ngắt)

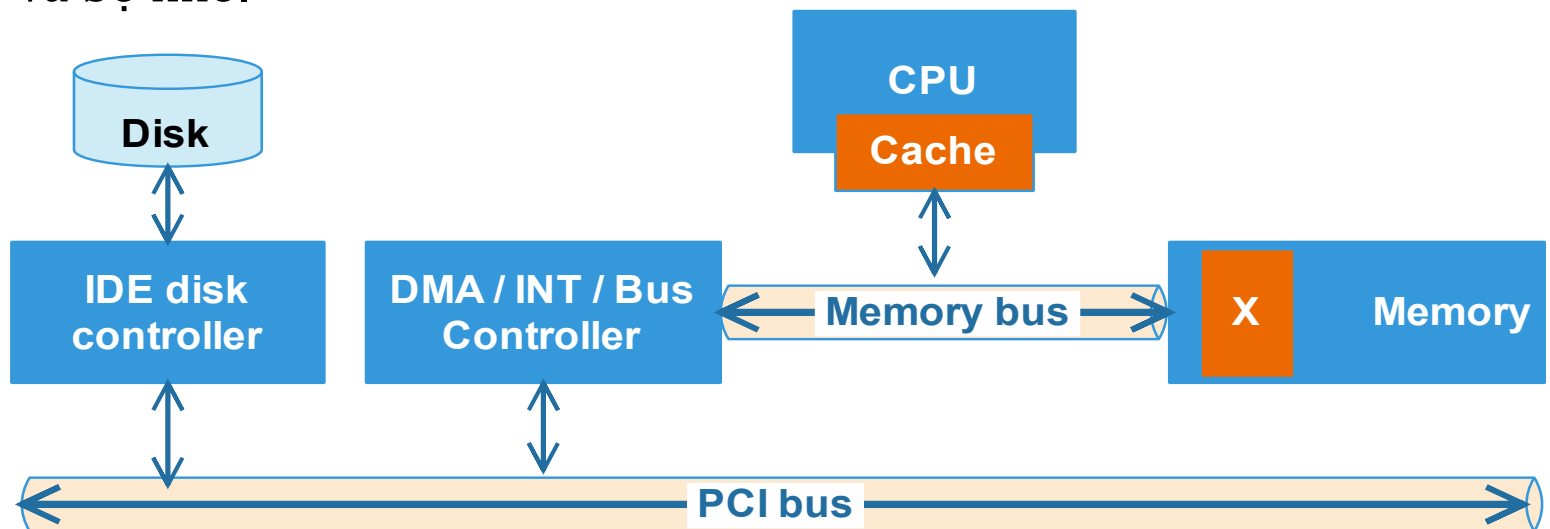
- ▶ *Programmable Interrupt Controller* (bộ điều khiển chương trình ngắt)
 - Tiếp nhận các **IRQ**, gửi tới CPU (tín hiệu **INT-R**)
 - Nhận hồi đáp từ CPU (tín hiệu **INT-A**) và chỉ thị chương trình ngắt tương ứng cho CPU xử lý.



Giao tiếp bằng chương trình

■ Dùng kỹ thuật DMA (*Direct Memory Access*)

- ▶ **DMA** – cơ chế hỗ trợ thiết bị truy cập bộ nhớ trực tiếp mà không cần trợ giúp của CPU.
- ▶ **DMA controller** (bộ điều khiển truy cập DMA):
 - CPU ghi khối lệnh truy xuất thiết bị vào **DMA controller** -> CPU làm việc khác.
 - **DMA controller** thay thế CPU điều khiển trao đổi dữ liệu giữa thiết bị và bộ nhớ.



Giao tiếp bằng chương trình

■ Dùng kỹ thuật DMA (*Direct Memory Access*)

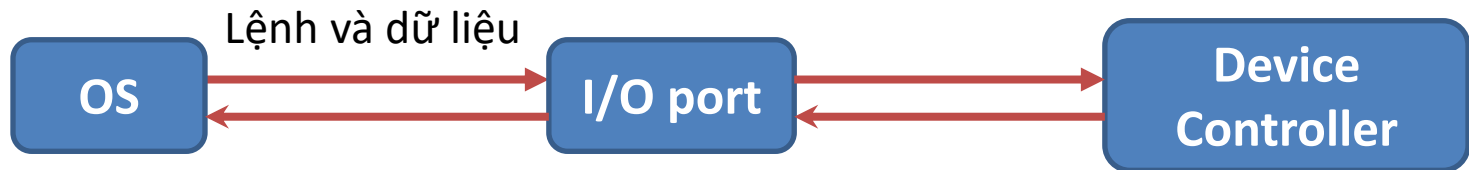
▶ Cơ chế hoạt động của DMA:

- 1) Sau khi nhận khối lệnh từ CPU, *DMA controller* chuyển lệnh (ví dụ: yêu cầu truyền **C** bytes dữ liệu vào vùng nhớ **X**) vào *Device Driver*.
- 2) *Device Driver* chuyển lệnh truy cập vào *Device Controller*
- 3) *Device Controller* bắt tay (*handshaking*) với *DMA controller*.
 - *Device Controller* gửi (*Request*) yêu cầu kết nối cho *DMA controller*.
 - *DMA controller* gửi trả xác nhận (*ACK*) kết nối cho *Device Controller*.
 - *Device Controller* trả xác nhận (*ACK*) kết nối cho *DMA controller*
- 4) *Device* truyền dữ liệu về *DMA controller*.
- 5) *DMA controller* chuyển dữ liệu vào vùng nhớ **X** cho đến khi **C=0**.
- 6) Khi **C=0**, *DMA controller* ngừng và báo hiệu hoàn thành cho CPU.

Giao tiếp bằng chương trình

■ Dùng địa chỉ *I/O port*:

- ▶ Dùng *I/O port* để đọc / ghi **trạng thái** của *controller*.
- ▶ Dùng *I/O port* để chuyển **lệnh** vào *thanh ghi lệnh* của *controller*.
- ▶ Dùng *I/O port* để chuyển **dữ liệu** vào *thanh ghi dữ liệu* của *controller*.



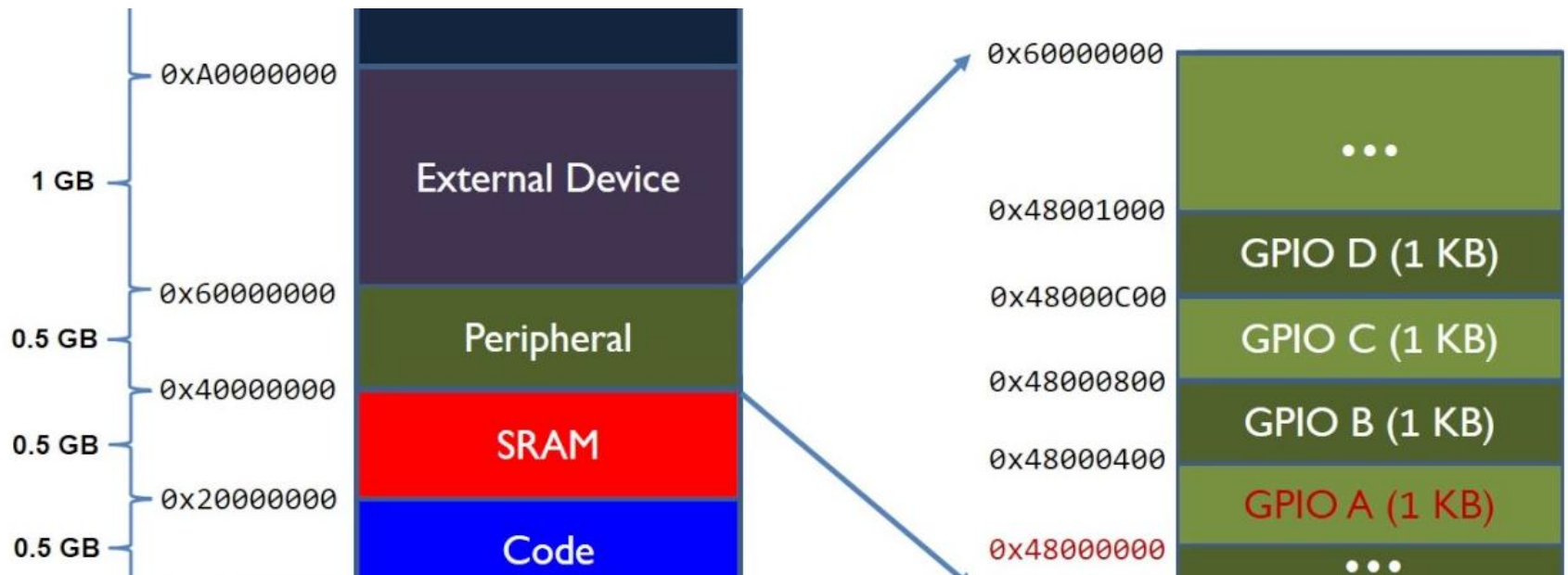
- ▶ Bảng địa chỉ port (*I/O address range*): chứa dãy địa chỉ port dùng giao tiếp với các thiết bị nhập/xuất.

I/O address range (hexadecimal)	device
2F8-2FF	serial port (secondary) controller
320-32F	hard-disk controller
378-37F	parallel port controller

Giao tiếp bằng chương trình

■ Dùng *Memory-mapped I/O*:

- Sử dụng khi giao tiếp với những thiết bị I/O có lượng RAM lớn như *VGA controller*.
- Ánh xạ *địa chỉ thiết bị I/O* vào *địa chỉ bộ nhớ* của hệ thống máy tính.
- Lập trình giao tiếp thiết bị dựa trên địa chỉ (đã ánh xạ) trong bộ nhớ.



I/O interface cho ứng dụng

■ Dẫn nhập I/O interface :

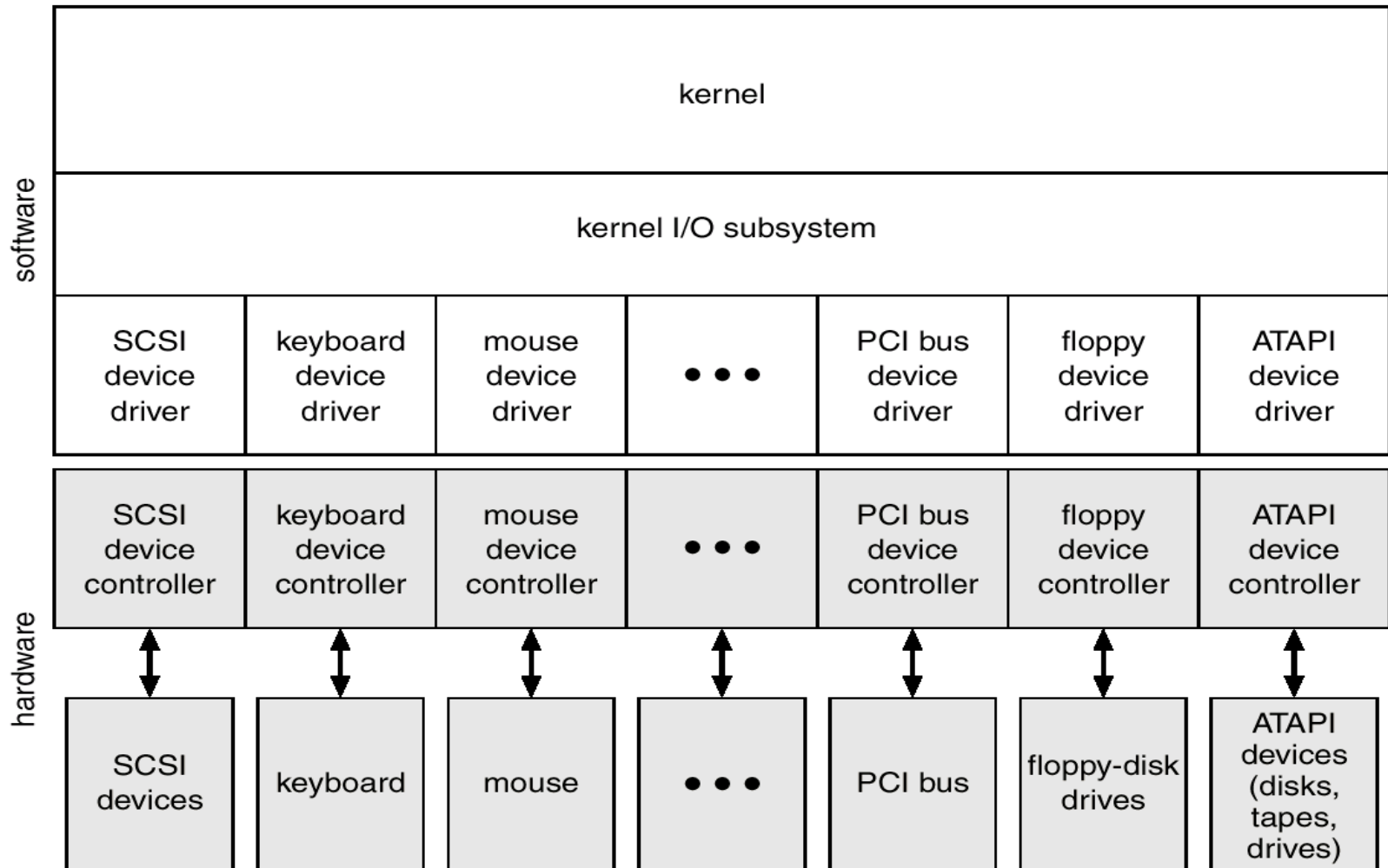
- ▶ Một hệ thống máy sẽ có nhiều thiết bị. Mỗi loại thiết bị có thể:
 - Có nhiều hãng sản xuất, nhiều dòng sản phẩm => có nhiều Driver khác nhau.
 - Có các kiểu giao tiếp khác nhau => lập trình giao tiếp khác nhau.
- ▶ Người lập trình không thể biết được hình thức giao tiếp, loại giao tiếp, driver... của thiết bị trên máy chạy ứng dụng.

■ *I/O interface*:

- ▶ Hệ điều hành xây dựng một *interface layer* cho phép ứng dụng giao tiếp thiết bị I/O mà không cần quan tâm loại thiết bị.
 - Lớp *I/O interface* sẽ nhận yêu cầu giao tiếp từ ứng dụng.
 - *I/O interface* sẽ chuyển đổi loại giao tiếp phù hợp với thiết bị => chuyển yêu cầu truy cập đến *Device driver* của thiết bị cụ thể.

I/O interface cho ứng dụng

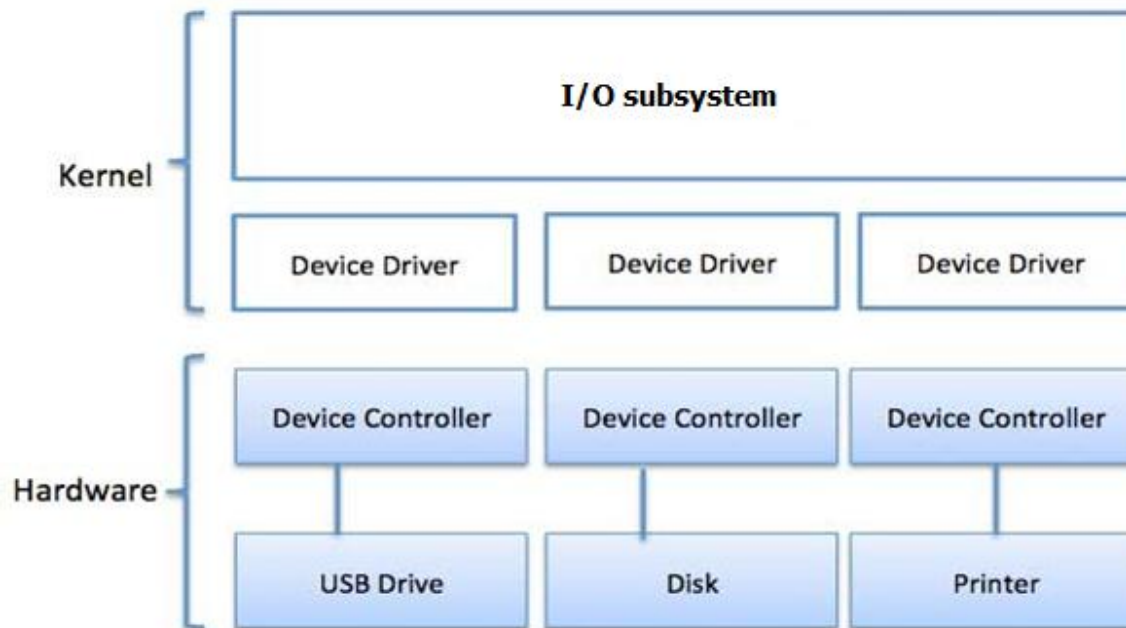
- I/O interface cho ứng dụng:



I/O subsystem của Hệ điều hành

■ I/O subsystem của Hệ điều hành:

- ▶ Là hệ thống quản lý, điều khiển thiết bị I/O của HĐH. Bao gồm:
 - Các **I/O system calls**: tiếp nhận các yêu cầu giao tiếp từ ứng dụng.
 - Lớp **I/O interface** cho các ứng dụng.
 - Quản lý tất cả **Device driver**.
 - Các vai trò khác nhằm điều phối, gia tăng hiệu suất truy cập thiết bị.



I/O subsystem của Hệ điều hành

■ Một số vai trò của I/O subsystem:

▶ **Schedule** (định thời) cho các *I/O requests*:

- Các *I/O requests* (yêu cầu truy xuất I/O của *process*) được đưa vào hàng đợi (**queue**) theo mỗi thiết bị.
- Điều phối các *I/O requests* theo thuật toán.
- Bảo đảm công bằng, hiệu suất cao.

▶ **Error handling** (Xử lý lỗi)

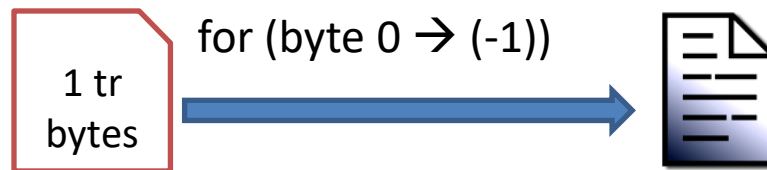
- *I/O subsystem* trả về mã lỗi khi yêu cầu I / O không thành công.
- Ghi nhật ký lỗi hệ thống (**event log**) , báo cáo sự cố.
- Thông tin cho tiến trình nếu thiết bị không khả dụng.

I/O subsystem của Hệ điều hành

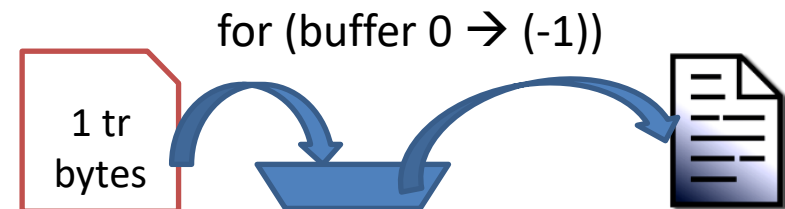
■ Một số vai trò của I/O subsystem:

▶ *Buffering* (đệm dữ liệu):

- Lưu dữ liệu tạm thời trong bộ nhớ nếu thiết bị I/O chưa sẵn sàng.
- Làm hàng đợi (*queue*) cho những thiết bị I/O không có bộ nhớ.
- Giải quyết vấn đề sai lệch tốc độ, kích thước dữ liệu khi thực hiện I/O



Vòng for chạy 1 triệu lần



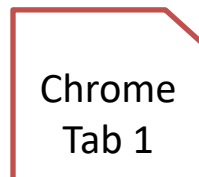
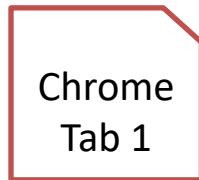
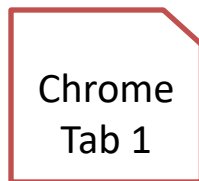
Vòng for chạy 1.000 lần

I/O subsystem của Hệ điều hành

■ Một số vai trò của I/O subsystem:

▶ *Caching*:

- Lưu tạm thời (*caching*) dữ liệu khi thực hiện I/O.
- Nếu các *I/O request* đến sau có yêu cầu loại dữ liệu đã *caching* => lấy dữ liệu trong cache cấp cho yêu cầu đó.
- *Caching* tăng hiệu suất.



Cache

Vòng for chạy 1.000 lần



for (buffer 0 → (-1))

I/O subsystem của Hệ điều hành

■ Một số vai trò của I/O subsystem :

▶ *Spooling*:

- Dữ liệu cần xuất (hay nhập) được đưa vào cái ống (*spool*).
- *I/O subsystem* chuyển dữ liệu dần dần từ *spool* ra thiết bị.
- *Printing spool* là một ứng dụng của *spooling*.

▶ *Error handling* (Xử lý lỗi)

- *I/O subsystem* trả về mã lỗi khi yêu cầu I / O không thành công
- Ghi nhật ký lỗi hệ thống (*event log*) , báo cáo sự cố.
- Thông tin cho tiến trình nếu thiết bị không khả dụng.

I/O interface cho ứng dụng

■ Cơ chế “I/O system call” đối với tiến trình:

▶ Cơ chế **Blocking**:

- Tiến trình sẽ bị **Suspended** khi thực hiện “*I/O System call*”.
- Khi “*I/O System call*” hoàn tất, tiến trình sẽ trở lại **Ready**.

▶ Cơ chế **Non-blocking**:

- Tiến trình được giữ **Running** trong khi thực hiện “*I/O System call*”.
- Cơ chế này chỉ có hiệu lực khi tiến trình thực hiện I/O dùng **Buffer** hoặc I/O qua **DMA**

▶ Cơ chế **Asynchronous**:

- Tiến trình sẽ về **Ready** khi thực hiện “*I/O System call*”.
- => tiến trình có thể được **Running** cả khi quá trình I/O chưa hoàn thành.
- => *I/O subsystem* sẽ báo hiệu cho *process* khi quá trình I/O hoàn tất.

Hiệu suất của I/O

■ Ảnh hưởng của I/O:

- ▶ Quá trình I/O gây ảnh hưởng lớn đến hiệu suất toàn hệ thống.
 - I/O chậm sẽ làm cho hiệu suất hệ thống giảm.
- ▶ Một số nguyên nhân có thể gây quá tải cho I/O:
 - Quá nhiều tác vụ chuyển ngữ cảnh do các *I/O interrupt*.
 - Mất nhiều thời gian khi gửi nhận dữ liệu giữa thiết bị và hệ thống.
 - CPU phải thực hiện quá nhiều lệnh điều khiển của *device driver*.
 - Các luồng dữ liệu mạng (*Network traffic*) tốc độ cao gây quá tải cho việc xử lý các *processes* liên quan.

Hiệu suất của I/O

- **Một số giải pháp tăng hiệu suất của I/O:**
 - ▶ Giảm số lần chuyển ngữ cảnh trong chương trình.
 - ▶ Áp dụng các giải pháp tăng tốc truy xuất dữ liệu:
 - Dùng kỹ thuật *buffering*, *caching*,...
 - ▶ Áp dụng các giải pháp giảm số lần ngắt quãng:
 - Tăng kích thước khối dữ liệu khi truyền.
 - Dùng kỹ thuật *spooling*
 - Dùng các bộ *controller* thông minh...
 - ▶ Sử dụng DMA nếu có thể.

Cám ơn !

